FML LAB RECORD

NAME : V.Gayathri Krishnaveni

REG NO . : 11259A513

SECTION : S9

SUBJECT : FUNDAMENTALS OF MACHINE LEARNING

EXPERIMENT-1: WORKING WITH NUMPY ARRAYS

Program:

1. NumPy program to print the NumPy version in your system. Code:

```
import numpy as np
print(np.__version__)
```

2. Numpy program to print different types of arrays.

#one-dimensional array

Code:

```
import numpy as np
x=np.arange(2,6).reshape(4)
print(x)
```

#two-dimensional array

Code:

```
import numpy as np
x=np.arange(2,10).reshape(2,4
) print(x)
```

#three-dimensional array

Code:

```
import numpy as np
x=np.arange(24).reshape(4,3,2
) print(x)
```

3. Numpy program to perform Array slicing.

Code:

```
import numpy as np
arr=np.array([1,2,3,4]
)
print(arr[1:3:2])
print(arr[:3])
print(arr[::2])
```

4. Numpy program to perform trigonometric functions.

Code:

```
import numpy as np
arr=np.array([0,30,60,90
]) print(np.sin(arr))
print(np.cos(arr))
```

```
print(np.tan(arr))
```

5.NumPy program to convert a list of numeric value into a one-dimensional NumPy array.
Code:

```
import numpy as np
l = [12.23, 13.32, 100, 36.32]
print("Original
List:",l) a =
np.array(l)
print("One-dimensional NumPy array: ",a)
```

6.NumPy program to create an array with values ranging from 12 to 38.
Code:

```
import numpy as
np x =
np.arange(12, 38)
print(x)
```

7.NumPy program to reverse an array (first element becomes last).
Code:

```
import numpy as
np import numpy
as np x = np.arange
(12, 38)
print("Original
array:") print(x)
print("Reverse
array:") x = x[::-1]
print(x)
```

8.NumPy program to append values to the end of an array.
Code:

```
import numpy as
np x = [10, 20, 30]
print("Original
array:") print(x)
```

```
x = np.append(x, [[40, 50, 60], [70, 80, 90]])
print("After append values to the end of the
array:") print(x)
```

9.NumPy program to find common values between two arrays.

Code:

```
import numpy as np
array1 = np.array([0, 10, 20, 40, 60])
print("Array1: ",array1)
array2 = [10, 30, 40]
print("Array2: ",array2)
print("Common values between two
arrays:") print(np.intersect1d(array1,
array2))
```

10. NumPy program to find the union of two arrays.

Code:

```
import numpy as np
array1 = np.array([0, 10, 20, 40, 60, 80])
print("Array1: ",array1)
array2 = [10, 30, 40, 50, 70]
print("Array2: ",array2)
print("Unique sorted array of values that are in either of the two input arrays:")
print(np.union1d(array1, array2))
```

11.NumPy program to concatenate two 2-dimensional arrays.

Code:

```
import numpy as np
a = np.array([[0, 1, 3], [5, 7, 9]])
b = np.array([[0, 2, 4], [6, 8, 10]])
c = np.concatenate((a, b), 1)
print(c)
```

12. Write a NumPy program to convert the values of Centigrade degrees into Fahrenheit degrees and vice versa.

```
#Fahrenheit degree to Centigrade degree.
```

Code:

```
import numpy as np

fvalues = [0, 12, 45.21, 34, 99.91, 32]

F = np.array(fvalues)

print("Values in Fahrenheit
degrees:") print(F)
print("Values in Centigrade
degrees:") print(np.round((5*F/9 -
5*32/9),2))
```

#Centigrade degree to Fahrenheit degree.

Code:

```
import numpy as np

cvalues = [-17.78, -11.11, 7.34, 1.11, 37.73, 0]

C = np.array(cvalues)

print("Values in Centigrade degrees:")
print(C)
print("Values in Fahrenheit
degrees:") print(np.round((9*C/5 +
32),2))
```

13.Numpy Program to perform arithmetic operations.

Code:

```
import numpy as np
a=np.arange(9,dtype=np.float_).reshape(3,3)
print("First Array:")
print(a)
b=np.array([10,20,30
]) print("Second
Array:") print(b)
print("Addition of Arrays:")
print(np.add(a,b))
print("Subtraction of Arrays:")
print(np.subtract(a,b))
print("Multiplication of
Arrays:")
print(np.multiply(a,b))
print("Division of Arrays:")
print(np.divide(a,b))
```

EX-2 : WORKING OF PANDAS DATAFRAME

Program:

1. **Pandas program to create and display a DataFrame from a specified dictionary data which has the index labels.**

Code:

```
import pandas
as pd import
numpy as np
exam_data = {'name': ['Anastasia', 'Dima', 'Katherine', 'James', 'Emily', 'Michael',
'Matthew', 'Laura', 'Kevin', 'Jonas'],
    'score': [12.5, 9, 16.5, np.nan, 9, 20, 14.5, np.nan, 8, 19],
    'attempts': [1, 3, 2, 3, 2, 3, 1, 1, 2, 1],
    'qualify': ['yes', 'no', 'yes', 'no', 'no', 'yes', 'yes', 'no', 'no', 'yes']}
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
df = pd.DataFrame(exam_data , index=labels)
print(df)
```

2. Pandas program to display a summary of the basic information about a specified DataFrame and its data.

Code:

```
df = pd.DataFrame(exam_data , index=labels)
print("Summary of the basic information about this DataFrame and its data:")
print(df.info())
```

3. Pandas program to get the first 3 rows of a given DataFrame.

Code:

```
df = pd.DataFrame(exam_data , index=labels)
print("First three rows of the data frame:")
print(df.iloc[:3])
```

4. Pandas program to select the 'name' and 'score' columns from the following DataFrame.

Code:

```
df = pd.DataFrame(exam_data ,
index=labels) print("Select specific
columns:") print(df[['name', 'score']]
```

5.Pandas program to select the rows where the number of attempts in the examination is greater than 2.

Code:

```
df = pd.DataFrame(exam_data , index=labels)

print("Number of attempts in the examination is greater than
2:") print(df[df['attempts'] > 2]
```

6.Pandas program to count the number of rows and columns of a DataFrame.

Code:

```
df = pd.DataFrame(exam_data ,

index=labels) total_rows=len(df.axes[0])

total_cols=len(df.axes[1])

print("Number of Rows: "+str(total_rows))
print("Number of Columns: "+str(total_cols))
```

7. Pandas program to select the rows where the score is missing, i.e. is NaN.

Code:

```
df = pd.DataFrame(exam_data , index=labels)
print("Rows where score is missing:")
print(df[df['score'].isnull()])
```

8. Pandas program to append a new row 'k' to DataFrame with given values for each column Now delete the new row and return the original data frame.

Code:

```
df = pd.DataFrame(exam_data , index=labels)
print("Original rows:")
print(df)

print("\nAppend a new row:")
df.loc['k'] = ['Suresh',15.5, 1,
'yes']
print("Print all records after insert a new
record:") print(df)
print("\nDelete the new row and display the original
rows:") df = df.drop('k')
print(df)
```

9. Pandas program to sort the DataFrame first by 'name' in descending order, then by 'score' in ascending order.

Code:

```
df = pd.DataFrame(exam_data , index=labels)
print("Orginal rows:")
print(df)

df.sort_values(by=['name', 'score'], ascending=[False, True])

print("Sort the data frame first by ‘name’ in descending order, then by ‘score’ in
```

ascending order:")
print(df)

9. **Pandas program to change the name 'James' to 'Suresh' in name column of the data frame.**
Code:

```
df = pd.                DataFrame(exam_data , index=labels)

                        print("Original rows:")
prin

prin                        printt("\nChange the name
'James' to ‘Shyam’:")

d                       f['name'] =
df['name'].replace('James', 'Shyam') print(df)
```

EX-3 : BASIC SCENARIO BASED PROGRAMS USING PYTHON FOR DATASCIENCE

1. **Student Marks Analysis Program**

```
marks = [78, 85, 92, 67, 88, 90, 76]
print("Highest Mark:", max(marks))
print("Lowest Mark:", min(marks))
print("Average Mark:", sum(marks)/len(marks))
```

2. Temperature Analysis Program

```
temperature = [30, 32, 31, 29, 35, 34, 33]
print("Maximum Temperature:", max(temperature))
print("Minimum Temperature:", min(temperature))
print("Average Temperature:", sum(temperature)/len(temperature))
```

3. Weekly Sales Analysis Program

```
sales = [2500, 3000, 2800, 3500, 4000, 3800, 4200]
print("Total Sales:", sum(sales))
print("Average Sales:", sum(sales)/len(sales))
```

4. Even and Odd Employee IDs Program

```
ids = [101, 102, 103, 104, 105, 106]
even = []
odd = []
for i in ids:
if i % 2 == 0:
even.append(i)
else:
odd.append(i)
print("Even IDs:", even)
print("Odd IDs:", odd)
```

5. Count Positive and Negative Values Program

```
values = [20, -5, 15, -10, 30, -2]
positive = 0
negative = 0
for i in values:
if i > 0:
positive += 1
elif i < 0:
```

```
negative += 1
 print("Positive Values:", positive)
    print("Negative Values:", negative)
```

6. Find Duplicate Customer IDs Program

```
      customer_ids = [101, 102, 103, 102, 104, 105, 101]
       duplicates = []
        for i in customer_ids:
        if customer_ids.count(i) > 1 and i not In duplicates:
         duplicates.append(i)
        print("Duplicate IDs:", duplicates)
```

7. Product Price Analysis Program

```
     prices = [250, 450, 120, 600, 350]
    print("Highest Price:", max(prices))
     print("Lowest Price:", min(prices)
```

8. Employee Salary Increment Program

```
       salary = [25000, 30000, 40000, 35000]
       new_salary = []
        for s in salary:
        new_salary.append(s * 1.10)
       print("Updated Salaries:", new_salary)
```

9. Attendance Percentage Program

```
          total_classes = 90
           attended = 81
          percentage = (attended / total_classes) * 100
           print("Attendance Percentage:", percentage)
```

10. Students Who Passed Program

```
  marks = [45, 67, 89, 32, 78, 55, 49]
  passed = []
  for m in marks:
  if m >= 50:
 passed.append(m)
 print("Passed Students Marks:", passed)
```

11. Shopping Bill Calculation Program

```
   prices = [120, 80, 150, 200]
     total = sum(prices)
 print("Total Bill:", total)
```

12. Word Frequency Analysis Program

```
feedback = "good service good quality excellent service"

words = feedback.split()
for word in set(words):
print(word, ":", words.count(word)).
```

13. Age Group Classification Program

```
ages = [12, 25, 45, 67, 15]
for age in ages:
if age < 18:
print(age, "Child")
elif age < 60:
print(age, "Adult")
else:
print(age, "Senior Citizen")
```

14. Electricity Bill Calculation Program

```
units = 250
if units <= 100:
bill = units * 2
elif units <= 200:
bill = 100 * 2 + (units - 100) * 3
else:
bill = 100 * 2 + 100 * 3 + (units - 200) * 5
print("Electricity Bill:", bill)
```

15. Monthly Expense Analysis Program

```
expenses = [12000, 15000, 18000, 11000, 17000]
highest = max(expenses)
month = expenses.index(highest) + 1
print("Highest Expense:", highest)
print("Month:", month)
```

EX-4 : PERFORM DATA PREPROCESSING ON GIVEN DATASET

Program

```
# Import libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
# Load dataset
df = pd.read_csv("Iris.csv")
# Display first 5 records
print("First 5 Records:")
print(df.head())
# Display dataset information
print("\nDataset Information:")
print(df.info())
# Check for missing values
print("\nMissing Values:")
print(df.isnull().sum())
# Remove the Id column (not useful for analysis)
df = df.drop("Id", axis=1)
# Encode the target column (Species)
encoder = LabelEncoder()
df["Species"] = encoder.fit_transform(df["Species"])
print("\nEncoded Species:")
print(df.head())
# Feature Scaling
scaler = StandardScaler()
features = ["SepalLengthCm", "SepalWidthCm",
"PetalLengthCm", "PetalWidthCm"]
df[features] = scaler.fit_transform(df[features])
```

```
print("\nPreprocessed Dataset:")
    print(df.head())
```

EX-5 : DATASET SPLITTING AND CROSS-VALIDATION USING IRIS DATASET

```
 # Dataset Splitting and Cross-Validation
# Using Iris Dataset

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
# ---------------------------------------------
# Step 1: Load the Iris Dataset
# ---------------------------------------------

iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Number of Samples:", len(X))
print("Number of Features:", X.shape[1])
print("Target Classes:", iris.target_names)

# ---------------------------------------------
# Step 2: Split Dataset into Training and Testing
# ---------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nDataset Splitting")
print("----------------")
print("Training samples:", len(X_train))
```

```
print("Testing samples :", len(X_test))

# ---------------------------------------------
# Step 3: Create Logistic Regression Model
# ---------------------------------------------

model = LogisticRegression(max_iter=200)

# Train the model
model.fit(X_train, y_train)

# ---------------------------------------------
# Step 4: Predict Test Data
# ---------------------------------------------

y_pred = model.predict(X_test)

# Calculate test accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nTest Set Performance")
print("--------------------")
print("Test Accuracy:", round(accuracy, 4))

# ---------------------------------------------
# Step 5: Implement 5-Fold Cross-Validation
# ---------------------------------------------

kf = KFold(
```

```
    n_splits=5,
    shuffle=True,
    random_state=42
)

cv_scores = cross_val_score(
    model,
    X,
    y,
    cv=kf,
    scoring='accuracy'
)

print("\n5-Fold Cross-Validation")
print("----------------------")

for i, score in enumerate(cv_scores, start=1):
    print("Fold", i, "Accuracy:", round(score, 4))

# ---------------------------------------------
# Step 6: Calculate Average Cross-Validation Score
# ---------------------------------------------

print("\nCross-Validation Results")
print("----------------------")
print("Average Accuracy:",
      round(cv_scores.mean(), 4))

print("Standard Deviation:",
```

```
round(cv_scores.std(), 4))
```

EX-6 : LINEAR REGRESSION USING THE IRIS DATASET

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import numpy as np

# ---------------------------------------------
# Step 1: Load Iris Dataset
# ---------------------------------------------

iris = load_iris()

X = iris.data
feature_names = iris.feature_names

# Predict Petal Length
# Column 2 = Petal Length
y = iris.data[:, 2]

# Use Sepal Length, Sepal Width and Petal Width
# as input features
X = iris.data[:, [0, 1, 3]]
```

```
print("Features:")
print(feature_names)

print("\nDataset Shape:", X.shape)
print("Target Shape:", y.shape)

# ----------------------------------------
# Step 2: Split Dataset
# ----------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))

# ----------------------------------------
# Step 3: Create Linear Regression Model
# ----------------------------------------

model = LinearRegression()

# ----------------------------------------
# Step 4: Train the Model
# ----------------------------------------

model.fit(X_train, y_train)
```

```
# ---------------------------------------------
# Step 5: Make Predictions
# ---------------------------------------------

y_pred = model.predict(X_test)

# ---------------------------------------------
# Step 6: Display Predictions
# ---------------------------------------------

print("\nActual vs Predicted Values")
print("-----------------------------")

for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print("Actual:", round(actual, 2),
          " Predicted:", round(predicted, 2))

# ---------------------------------------------
# Step 7: Evaluate the Model
# ---------------------------------------------

mse = mean_squared_error(y_test, y_pred)

rmse = np.sqrt(mse)

mae = mean_absolute_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

print("\nModel Evaluation")
print("--------------------------")
print("Mean Squared Error (MSE):", round(mse, 4))
```

```
print("Root Mean Squared Error (RMSE):", round(rmse, 4))
print("Mean Absolute Error (MAE):", round(mae, 4))
print("R2 Score:", round(r2, 4))

# ---------------------------------------------
# Step 8: Display Model Coefficients
# ---------------------------------------------

print("\nModel Coefficients")
print("---------------------------")

for name, coefficient in zip(
        ["Sepal Length", "Sepal Width", "Petal Width"],
        model.coef_):

    print(name, ":", round(coefficient, 4))

print("Intercept:", round(model.intercept_, 4))
```

EX-7 : KNN CLASSIFICATION USING IRIS DATASET

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
    recall_score,
    f1_score
)

# ---------------------------------------------
# Step 1: Load Iris Dataset
# ---------------------------------------------

iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Number of Samples:", len(X))
print("Number of Features:", X.shape[1])
print("Target Classes:", iris.target_names)

# ---------------------------------------------
# Step 2: Split Dataset
# ---------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nDataset Splitting")
print("----------------")
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

# ---------------------------------------------
# Step 3: Feature Scaling
# ---------------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------------------------------------------
# Step 4: Create KNN Model
# ---------------------------------------------

knn = KNeighborsClassifier(n_neighbors=5)

# ---------------------------------------------
# Step 5: Train the Model
# ---------------------------------------------

knn.fit(X_train, y_train)
```

```
# ---------------------------------------------
# Step 6: Make Predictions
# ---------------------------------------------

y_pred = knn.predict(X_test)

# ---------------------------------------------
# Step 7: Calculate Evaluation Metrics
# ---------------------------------------------

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    average='weighted'
)

recall = recall_score(
    y_test,
    y_pred,
    average='weighted'
)

f1 = f1_score(
    y_test,
    y_pred,
    average='weighted'
)
```

```
# ---------------------------------------------
# Step 8: Display Results
# ---------------------------------------------

print("\nKNN Model Evaluation")
print("--------------------")

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))

print("\nConfusion Matrix")
print("----------------")
print(cm)
```

EX-8 : SVM CLASSIFICATION USING IRIS DATASET

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    precision_score,
```

```
    recall_score,
    f1_score
)

# ------------------------------------------------
# Step 1: Load Iris Dataset
# ------------------------------------------------

iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Number of Samples:", len(X))
print("Number of Features:", X.shape[1])
print("Target Classes:", iris.target_names)

# ------------------------------------------------
# Step 2: Split Dataset
# ------------------------------------------------

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nDataset Splitting")
print("-----------------")
```

```
print("Training samples:", len(X_train))
print("Testing samples :", len(X_test))

# ---------------------------------------------
# Step 3: Feature Scaling
# ---------------------------------------------

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------------------------------------------
# Step 4: Create SVM Model
# ---------------------------------------------

model = SVC(
    kernel='linear',
    C=1.0
)

# ---------------------------------------------
# Step 5: Train the Model
# ---------------------------------------------

model.fit(X_train, y_train)

# ---------------------------------------------
# Step 6: Make Predictions
# ---------------------------------------------

y_pred = model.predict(X_test)
```

```
# ---------------------------------------------
# Step 7: Calculate Evaluation Metrics
# ---------------------------------------------

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

precision = precision_score(
    y_test,
    y_pred,
    average='weighted'
)

recall = recall_score(
    y_test,
    y_pred,
    average='weighted'
)

f1 = f1_score(
    y_test,
    y_pred,
    average='weighted'
)

# ---------------------------------------------
# Step 8: Display Results
# ---------------------------------------------

print("\nSVM Model Evaluation")
```

```
print("--------------------")

print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1-Score :", round(f1, 4))

print("\nConfusion Matrix")
print("----------------")
print(cm)
```

EX-9

```
                # K-Means Clustering using Iris Dataset

from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# ---------------------------------------------
# Step 1: Load Iris Dataset
# ---------------------------------------------

iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Shape:", X.shape)
print("Number of Samples:", X.shape[0])
print("Number of Features:", X.shape[1])

print("\nFeatures:")
print(iris.feature_names)

# ---------------------------------------------
# Step 2: Feature Scaling
# ---------------------------------------------

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# ---------------------------------------------
# Step 3: Create K-Means Model
# ---------------------------------------------

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

# ---------------------------------------------
# Step 4: Fit K-Means Model
# ---------------------------------------------

kmeans.fit(X_scaled)
```

```
# ---------------------------------------------
# Step 5: Get Cluster Labels
# ---------------------------------------------

labels = kmeans.labels_

# ---------------------------------------------
# Step 6: Get Cluster Centroids
# ---------------------------------------------

centroids = kmeans.cluster_centers_

# ---------------------------------------------
# Step 7: Display Results
# ---------------------------------------------

print("\nCluster Labels:")
print(labels)

print("\nCluster Centroids:")
print(centroids)

# ---------------------------------------------
# Step 8: Calculate Silhouette Score
# ---------------------------------------------

silhouette = silhouette_score(X_scaled, labels)

print("\nCluster Evaluation")
print("-----------------")
print("Silhouette Score:",
     round(silhouette, 4))

# ---------------------------------------------
# Step 9: Display Cluster Count
# ---------------------------------------------

print("\nNumber of Samples in Each Cluster:")

for i in range(3):
    print(
        "Cluster", i,
        ":",
        sum(labels == i),
        "samples"
    )

# ---------------------------------------------
# Step 10: Visualize Clusters
```

```
# ------------------------------------------------

plt.figure(figsize=(8, 6))

plt.scatter(
    X_scaled[:, 2],
    X_scaled[:, 3],
    c=labels
)

plt.scatter(
    centroids[:, 2],
    centroids[:, 3],
    marker='X',
    s=200
)

plt.xlabel("Petal Length")
plt.ylabel("Petal Width")
plt.title("K-Means Clustering on Iris Dataset")

plt.show()
```

EX-10 : HIERARCHICAL CLUSTERING USING IRIS DATSET

```
# Hierarchical Clustering using Iris Dataset

import matplotlib.pyplot as plt

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score

from scipy.cluster.hierarchy import dendrogram, linkage

# ---------------------------------------------
# Step 1: Load Iris Dataset
# ---------------------------------------------
```

```
   iris = load_iris()

   X = iris.data
      y = iris.target

 print("Dataset Shape:", X.shape)
 print("Number of Samples:", X.shape[0])
print("Number of Features:", X.shape[1])

print("\nFeatures:")
print(iris.feature_names)

# ------------------------------------------------
       # Step 2: Feature Scaling
   # ------------------------------------------------

  scaler = StandardScaler()
     X_scaled = scaler.fit_transform(X)

         # ------------------------------------------------
            # Step 3: Create Dendrogram
         # ------------------------------------------------

            linked = linkage(X_scaled, method='ward')

                plt.figure(figsize=(10, 6))

                       dendrogram(
                          linked,
              truncate_mode='lastp',
                   p=20
                     )
```

```
        plt.title("Hierarchical Clustering Dendrogram")
    plt.xlabel("Data Points / Clusters")
    plt.ylabel("Distance")

        plt.show()

    # ------------------------------------------------
      # Step 4: Apply Agglomerative Clustering
   # ------------------------------------------------

  model = AgglomerativeClustering(
     n_clusters=3,
    linkage='ward'
     )

   labels = model.fit_predict(X_scaled)

  # ------------------------------------------------
  # Step 5: Display Cluster Labels
   # ------------------------------------------------

  print("\nCluster Labels:")
 print(labels)

# ------------------------------------------------
# Step 6: Calculate Silhouette Score
     # ------------------------------------------------

 silhouette = silhouette_score(
    X_scaled,
      labels
```

```
            )

  print("\nSilhouette Score:",
round(silhouette, 4))

        # ---------------------------------------------
           # Step 7: Count Samples in Each Cluster
       # ---------------------------------------------

         print("\nNumber of Samples in Each Cluster:")

              for i in range(3):
                   print(
             "Cluster", i,
                  ":",
             sum(labels == i),
                   "samples"
                        )

                # ---------------------------------------------
                 # Step 8: Visualize Clusters
               # ---------------------------------------------

                plt.figure(figsize=(8, 6))

                    plt.scatter(
               X_scaled[:, 2],
                X_scaled[:, 3],
               c=labels
                    )

        plt.xlabel("Petal Length")
```

```
    plt.ylabel("Petal Width")
plt.title("Hierarchical Clustering on Iris Dataset")

            plt.show()
```

EX-11 : IMPLEMENT RANDOM FOREST AND ANALYZE PERFORMANCE

```
# Random Forest Classification using Iris Dataset

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

# 3. Create Random Forest model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# 4. Train the model
model.fit(X_train, y_train)

# 5. Make predictions
y_pred = model.predict(X_test)

# 6. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Random Forest Classification")
print("----------------------------")
print("Number of trees:", 100)
```

```
print("Accuracy:", accuracy)

# 7. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# 8. Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# 9. Display Confusion Matrix
plt.figure(figsize=(6, 4))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Random Forest - Confusion Matrix")
plt.show()
```

EX-12 : IMPLEMENT BOOSTING TECHNIQUES AND COMPARE RESULTS

```
# Implement Boosting Techniques and Compare Results
# Dataset: Iris

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
import matplotlib.pyplot as plt

# 1. Load dataset
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
 test_size=0.20,
  random_state=42,
   stratify=y
 )

# 3. Create AdaBoost model
 ada_model = AdaBoostClassifier(
 n_estimators=100,
  random_state=42
 )

 # 4. Create Gradient Boosting model
  gb_model = GradientBoostingClassifier(
   n_estimators=100,
       random_state=42
 )

          # 5. Train models
     ada_model.fit(X_train, y_train)
     gb_model.fit(X_train, y_train)

           # 6. Predictions
      ada_pred = ada_model.predict(X_test)
         gb_pred = gb_model.predict(X_test)

    # 7. Function to calculate performance
 def evaluate_model(name, y_test, y_pred):
     accuracy = accuracy_score(y_test, y_pred)
          precision = precision_score(
     y_test, y_pred, average="weighted"
```

```
        )
    recall = recall_score(
y_test, y_pred, average="weighted"
        )
    f1 = f1_score(
    y_test, y_pred, average="weighted"
        )

  print("\n", name)
      print("-" * 30)
print("Accuracy :", round(accuracy, 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
 print("F1-Score :", round(f1, 4))

  return accuracy, precision, recall, f1



     # 8. Evaluate AdaBoost
 ada_results = evaluate_model(
    "AdaBoost",
       y_test,
      ada_pred
     )

# 9. Evaluate Gradient Boosting
 gb_results = evaluate_model(
  "Gradient Boosting",
   y_test,
  gb_pred
    )
```

```
# 10. Compare accuracy
models = ["AdaBoost", "Gradient Boosting"]
      accuracies = [
  ada_results[0],
   gb_results[0]
  ]

plt.figure(figsize=(7, 5))
 plt.bar(models, accuracies)




 plt.xlabel("Boosting Technique")
  plt.ylabel("Accuracy")
plt.title("Comparison of Boosting Techniques")

plt.ylim(0, 1.1)
   plt.show()
```

EX-13: IMPLEMENT A BASIC REINFORCEMENT LEARNING MODEL

```
import numpy as np
import random

# Number of states and actions
num_states = 5
num_actions = 2

# Actions
# 0 = Left
# 1 = Right

# Initialize Q-table
Q = np.zeros((num_states, num_actions))
```

```
# Parameters
alpha = 0.1       # Learning rate
gamma = 0.9       # Discount factor
epsilon = 0.2     # Exploration probability
episodes = 1000

# Environment function
def get_next_state(state, action):

    if action == 0:       # Move Left
        next_state = max(0, state - 1)
    else:
        # Move Right
        next_state = min(num_states - 1, state + 1)

    # Reward
    if next_state == 4:
        reward = 10
    else:
        reward = -1

    return next_state, reward



# Q-Learning
for episode in range(episodes):

state = 0

while state != 4:

  # Epsilon-greedy action selection
```

```
    if random.random() < epsilon:
        action = random.randint(0, 1)
    else:
        action = np.argmax(Q[state])

    # Perform action
    next_state, reward = get_next_state(state, action)

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state


        #  Display Q-table
      print("Learned Q-Table:")
  print(Q)


        # Test the learned policy
    state = 0
    path = [state]

     while state != 4:

action = np.argmax(Q[state])

next_state, reward = get_next_state(state, action)
```

```
state = next_state
path.append(state)

        print("\nLearned Path:")
              print(path)

            print("\nOptimal Policy:")
       for state in range(num_states - 1):

action = np.argmax(Q[state])

if action == 0:
   print("State", state, "-> Left")
else:
   print("State", state, "-> Right")
```

EX-14 : BUILD A NEURAL NETWORK USING TENSORFLOW/ KERAS

```
# Build a Neural Network using TensorFlow/Keras
# Dataset: Iris
```

```
import numpy as np
Import tensorflow as tf

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report

# 1. Load Iris dataset
iris = load_iris()

X = iris.data
y = iris.target

# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
    )

# 3. Standardize features
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 4. Build neural network
```

```
model = Sequential([
Dense(16, activation='relu', input_shape=(4,)),
  Dense(8, activation='relu'),
  Dense(3, activation='softmax')
    ])

            # 5. Compile model
             model.compile(
              optimizer='adam',
        loss='sparse_categorical_crossentropy',
              metrics=['accuracy']
              )

        # 6. Display model architecture
             model.summary()

           # 7. Train the model
          history = model.fit(
                  X_train,
                     y_train,
                  epochs=50,
                batch_size=8,
           validation_split=0.2,
                     verbose=1
                   )

                # 8. Evaluate model
      loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

                          print("\nTest Loss:", round(loss, 4))
                       print("Test Accuracy:", round(accuracy, 4))
```

```
# 9. Make predictions
y_probability = model.predict(X_test, verbose=0)

y_pred = np.argmax(y_probability, axis=1)

# 10. Classification report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```